

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ
MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN
PHÁT TRIỂN ỨNG DỤNG
INTERNET OF THINGS (CO3037)

– NHÓM SINH VIÊN THỰC HIỆN (SVTH):
LÊ NHẬT MINH - 2212046
LA ANH TÚ - 2112601
GITHUB LINK: [minh-le1710/CO3037-2](https://github.com/minh-le1710/CO3037-2)

TP. Hồ Chí Minh, Tháng 12 / 2025

Mục lục

1	Giới thiệu	3
1.1	Ảnh xạ task và nhánh mã nguồn	3
1.2	Mục tiêu tổng quát của hệ thống	3
2	Task 1: RTOS và LED cảnh báo theo nhiệt độ	4
2.1	Mục tiêu	4
2.2	Phần cứng và luật điều khiển	4
2.3	Thiết kế firmware	4
2.4	Kết quả và nhận xét	5
3	Task 2: NeoPixel hiển thị độ ẩm	5
3.1	Mục tiêu	5
3.2	Nguyên lý hiển thị	5
3.3	Thiết kế firmware	6
3.4	Nhận xét	6
4	Task 3: Hiển thị trạng thái môi trường trên LCD I2C	6
4.1	Mục tiêu	6
4.2	Luật phân loại	7
4.3	Kiến trúc đồng bộ	7
4.4	Kết quả	8
5	Task 4: Web server điều khiển thiết bị qua WiFi AP	8
5.1	Mục tiêu	8
5.2	Thiết kế mạng và giao diện	8
5.3	Luồng xử lý HTTP	9
5.4	Đánh giá	9
6	Task 5: TinyML - huấn luyện, đánh giá và nhúng mô hình	9
6.1	Mục tiêu	9
6.2	Mô tả dataset và mô hình	10
6.3	Kết quả huấn luyện	10
6.4	Tích hợp vào firmware	11
6.5	Nhận xét	11
7	Task 6: Publish dữ liệu lên CoreIOT Cloud Server	11
7.1	Mục tiêu	11
7.2	Kiến trúc phần mềm	12
7.3	Access Point, Station và dashboard	12
7.4	Telemetry gửi lên CoreIOT	12
7.5	Kết quả thực nghiệm	13



7.6	Đánh giá	13
8	Kết luận	14

1 Giới thiệu

Báo cáo trình bày quá trình xây dựng một hệ thống giám sát và điều khiển môi trường trên nền vi điều khiển ESP32-S3. Hệ thống được phát triển theo 6 task tăng dần về mức độ hoàn thiện, bắt đầu từ việc đọc cảm biến và điều khiển LED, sau đó mở rộng sang hiển thị trực quan, web dashboard, suy luận TinyML và cuối cùng là publish dữ liệu lên nền tảng CoreIOT Cloud qua MQTT.

Trong quá trình thực hiện, mã nguồn được tổ chức theo từng nhánh tương ứng với từng task. Ở các mốc khác nhau, phần cứng có sự thay đổi giữa ESP32-S3 + DHT22 + OLED và Yolo UNO (ESP32-S3) + DHT20 + LCD I2C. Vì vậy nội dung báo cáo được trình bày bám theo **nhánh thực hiện của từng task** để phản ánh đúng cách triển khai thực tế.

1.1 Ánh xạ task và nhánh mã nguồn

Task	Nhánh / tài liệu	Nội dung chính
Task 1	bt1_iot_credit-1	RTOS + semaphore + LED nháy theo dải nhiệt độ
Task 2	credit-2	NeoPixel hiển thị độ ẩm theo thanh 10 mức
Task 3	bt1_iot_credit_3	LCD I2C hiển thị trạng thái môi trường
Task 4	credit-4	Web server ở chế độ Access Point để điều khiển thiết bị
Task 5	bt1_iot_credit_5, docs/task5_tinymml.md	Huấn luyện, nhúng và đánh giá TinyML trên thiết bị
Task 6	bt1_iot_credit_6, docs/task6_coreiot.md	Publish telemetry lên CoreIOT Cloud Server

1.2 Mục tiêu tổng quát của hệ thống

Chuỗi bài tập lớn được triển khai theo hướng tăng dần độ phức tạp:

- Bắt đầu từ một tác vụ RTOS đơn giản: đọc cảm biến và điều khiển LED theo điều kiện nhiệt độ.
- Bổ sung khả năng hiển thị mức độ ẩm qua NeoPixel để tăng tính trực quan.
- Nâng lên hiển thị trạng thái tổng quát NORMAL / WARNING / CRITICAL trên thiết bị ngoại vi I2C.
- Mở rộng sang giao tiếp mạng cục bộ qua WiFi Access Point và web dashboard.
- Tích hợp TinyML để thay thế phân loại thủ công bằng mô hình suy luận nhúng.
- Hoàn thiện theo định hướng IoT thực thụ bằng cách đẩy telemetry lên cloud.

2 Task 1: RTOS và LED cảnh báo theo nhiệt độ

2.1 Mục tiêu

Task 1 yêu cầu xây dựng một cơ chế cảnh báo nhiệt độ đơn giản nhưng có tổ chức đúng tinh thần hệ điều hành thời gian thực:

- Đọc cảm biến nhiệt độ, độ ẩm DHT20 theo chu kỳ.
- Phân loại nhiệt độ thành ba vùng COOL / NORMAL / HOT.
- Điều khiển một LED duy nhất bằng ba kiểu nháy khác nhau tương ứng từng vùng nhiệt độ.
- Dùng **binary semaphore** để đồng bộ giữa task đọc cảm biến và task nháy LED.

2.2 Phần cứng và luật điều khiển

Ở nhánh `bt1_iot_credit-1`, hệ thống dùng bo Yolo UNO với cảm biến DHT20 qua giao tiếp I2C (`SDA = GPIO11`, `SCL = GPIO12`) và LED điều khiển tại `GPIO48`. Nhiệt độ được chia thành ba dải như ở Bảng 1.

Bảng 1: Các vùng nhiệt độ và mẫu nháy LED của Task 1

Vùng	Hành vi LED
COOL, $T < 24^{\circ}C$	Nháy chậm: bật 1 s, tắt 1 s
NORMAL, $24^{\circ}C \leq T < 29^{\circ}C$	Nháy đôi: 180 ms bật, 180 ms tắt, 180 ms bật, 900 ms tắt
HOT, $T \geq 29^{\circ}C$	Nháy nhanh: 120 ms bật, 120 ms tắt

2.3 Thiết kế firmware

Kiến trúc RTOS của Task 1 gồm hai task chính:

- `sensorTask`: đọc DHT20 mỗi 2 giây, phân loại dải nhiệt độ, cập nhật vùng dữ liệu dùng chung.
- `ledTask`: chờ thông báo từ task cảm biến rồi chạy pattern nhấp nháy tương ứng.

Hai semaphore được dùng đúng mục đích:

- `stateMutex`: bảo vệ dữ liệu dùng chung `latestTemperatureC`, `latestHumidityPercent`, `behavior`, `hasReading`.
- `behaviorChanged`: binary semaphore dùng như tín hiệu sự kiện, chỉ được `give` khi vùng nhiệt độ thay đổi.

Điểm đáng giá trong cách cài đặt là `ledTask` không delay “mù” toàn bộ một chu kỳ nhảy. Thay vào đó, ở từng khoảng bật/tắt LED, task dùng `xSemaphoreTake(..., timeout)` để chờ ngắt quãng. Nhờ vậy nếu nhiệt độ chuyển vùng trong lúc LED đang nhảy, pattern cũ sẽ bị ngắt gần như tức thời và chuyển sang pattern mới mà không cần polling liên tục.

2.4 Kết quả và nhận xét

Task 1 đáp ứng đúng yêu cầu của đề:

- Có ba kiểu nhảy LED khác nhau theo nhiệt độ.
- Có tách task rõ ràng giữa đo cảm biến và điều khiển hiển thị.
- Có dùng semaphore đúng vai trò đồng bộ, không chỉ dùng delay tuần tự.

So với một chương trình `loop()` truyền thống, phiên bản RTOS này linh hoạt hơn rõ rệt vì có thể mở rộng tiếp sang Task 2, Task 3 và các chức năng mạng mà không phải viết lại lõi điều khiển.

3 Task 2: NeoPixel hiển thị độ ẩm

3.1 Mục tiêu

Task 2 mở rộng hệ thống bằng một cơ chế hiển thị trực quan cho độ ẩm môi trường:

- Tận dụng lại dữ liệu độ ẩm từ cảm biến môi trường.
- Quy đổi độ ẩm 0–100% thành thanh hiển thị 10 mức.
- Dùng màu sắc để giúp nhận biết nhanh mức độ ẩm thấp, trung bình hay cao.
- Tổ chức logic hiển thị bằng một task riêng, đồng bộ với task cảm biến qua semaphore.

3.2 Nguyên lý hiển thị

Trong nhánh `credit-2`, độ ẩm được ánh xạ thành số LED sáng trên dải NeoPixel. Hàm `showHumidityBar(hum)` quy đổi:

$$\text{level} = \text{round} \left(\frac{\text{hum}}{100} \times 10 \right)$$

sau đó giới hạn giá trị trong đoạn từ 0 đến 10.

Bảng 2: Quy ước màu của dải NeoPixel ở Task 2

Nhóm LED	Ý nghĩa
LED 1–3 màu xanh lá	Mức ẩm thấp hoặc vùng dễ chịu
LED 4–7 màu vàng	Độ ẩm trung bình, cần theo dõi
LED 8–10 màu đỏ	Độ ẩm cao, có nguy cơ bí hoặc ẩm mốc

3.3 Thiết kế firmware

Task 2 vẫn giữ `sensorTask` làm nơi đọc dữ liệu gốc, nhưng phần hiển thị được tách riêng qua:

- `xHumSemaphore`: task cảm biến `give` mỗi khi có mẫu độ ẩm hợp lệ mới.
- `vTaskNeoPixel`: chờ semaphore, đọc `g_humidity` rồi cập nhật thanh LED.

Ưu điểm của cách làm này là:

- Phần điều khiển NeoPixel không làm chậm chu kỳ đọc cảm biến.
- Dễ thay đổi logic hiển thị mà không ảnh hưởng Task 1.
- Dễ tích hợp tiếp với các task hiển thị và TinyML ở các mốc sau.

3.4 Nhận xét

Task 2 cho thấy cách dùng RTOS không chỉ phù hợp với xử lý song song mà còn giúp **phân tách trách nhiệm** rất rõ ràng: một task chịu trách nhiệm thu thập dữ liệu, task còn lại chuyên trách hiển thị. Về mặt trải nghiệm, thanh NeoPixel mang lại cảm giác “đọc nhanh bằng mắt” tốt hơn nhiều so với việc chỉ xem số độ ẩm trên serial monitor.

4 Task 3: Hiển thị trạng thái môi trường trên LCD I2C

4.1 Mục tiêu

Task 3 nâng cấp mức hiển thị từ “giá trị riêng lẻ” sang “trạng thái hệ thống”. Ở nhánh `bt1_iot_credit_3`, hệ thống dùng LCD I2C 16x2 để thông báo ba trạng thái:

- NORMAL
- WARNING
- CRITICAL

Điểm đáng chú ý là trong nhánh này, chương trình tổ chức ba “kênh trạng thái” riêng biệt bằng queue và semaphore, thay vì chỉ cập nhật hiển thị theo một luồng xử lý duy nhất.

4.2 Luật phân loại

Mỗi mẫu đo gồm nhiệt độ và độ ẩm được đưa qua hàm `classifyReading(temp, hum)`. Các ngưỡng sử dụng trong nhánh này được cho ở Bảng 3.

Bảng 3: Ngưỡng phân loại ở Task 3

Trạng thái	Điều kiện
CRITICAL	$T < 20^{\circ}C$ hoặc $T \geq 32^{\circ}C$, hoặc $H < 30\%$ hoặc $H > 80\%$
WARNING	Không thuộc CRITICAL và thỏa $T < 23^{\circ}C$ hoặc $T \geq 29^{\circ}C$, hoặc $H < 40\%$ hoặc $H > 70\%$
NORMAL	Các trường hợp còn lại

4.3 Kiến trúc đồng bộ

Task 3 không chỉ có một semaphore như các task trước, mà tổ chức thành các thành phần sau:

- **sensorTask**: đọc DHT20, phân loại trạng thái, gửi mẫu đo vào queue tương ứng.
- **normalChannel**, **warningChannel**, **criticalChannel**: mỗi kênh có một queue kích thước 1 và một binary semaphore.
- **displayTask**: được tạo thành ba instance, mỗi instance phụ trách một trạng thái.
- **lcdMutex**: mutex bảo vệ LCD, tránh hai task cùng ghi I2C tại một thời điểm.

Mỗi khi có dữ liệu mới, **sensorTask** sẽ chọn đúng kênh đích bằng `selectChannel()`, dùng `xQueueOverwrite()` để giữ lại mẫu mới nhất, rồi **give** semaphore cho task hiển thị tương ứng. Cách làm này giúp hệ thống:

- Không phải tự kiểm tra trạng thái liên tục trong task hiển thị.
- Tránh backlog vì mỗi trạng thái chỉ giữ mẫu mới nhất.
- Đảm bảo LCD luôn phản ánh đúng điều kiện hiện thời.

4.4 Kết quả

Task 3 biến hệ thống từ dạng “có cảm biến và LED” thành một thiết bị giám sát môi trường hoàn chỉnh hơn:

- Dòng 1 của LCD hiển thị nhiệt độ.
- Dòng 2 hiển thị độ ẩm và chú thích trạng thái như OK, CHECK, ALERT.
- Việc tách ba kênh trạng thái giúp code sạch, rõ vai trò và dễ mở rộng.

5 Task 4: Web server điều khiển thiết bị qua WiFi AP

5.1 Mục tiêu

Task 4 đưa hệ thống từ mức hiển thị cục bộ sang điều khiển không dây trong mạng nội bộ:

- ESP32-S3 phát WiFi ở chế độ **Access Point**.
- Tạo một web server để người dùng truy cập qua điện thoại hoặc máy tính.
- Điều khiển hai thiết bị số từ giao diện web.
- Duy trì đồng thời các task của cảm biến, LED, NeoPixel và hiển thị trạng thái.

5.2 Thiết kế mạng và giao diện

Ở nhánh `credit-4`, ESP32-S3 khởi tạo:

- SSID: ESP32_S3_AP
- Mật khẩu: 12345678
- Web server cổng 80

Hai thiết bị mẫu được gán vào:

- DEVICE1_PIN = 2
- DEVICE2_PIN = 4

Giao diện HTML/CSS được thiết kế theo dạng mini dashboard, gồm hai thẻ điều khiển và hai cặp nút **Turn ON**/**Turn OFF**. Phần hiển thị trạng thái dùng badge **ON/OFF** để người dùng quan sát nhanh.

5.3 Luồng xử lý HTTP

Web server cung cấp các route chính:

- `/`: render trang điều khiển.
- `/device1/on`, `/device1/off`: bật/tắt thiết bị 1.
- `/device2/on`, `/device2/off`: bật/tắt thiết bị 2.

Khi người dùng bấm nút:

- server cập nhật `device1State` hoặc `device2State`,
- gọi `applyDeviceState()` để xuất mức logic xuống GPIO,
- trả mã 303 See Other để trình duyệt tự quay lại trang chính.

Trong khi đó, vòng `loop()` vẫn chỉ làm hai việc rất ngắn:

- `server.handleClient()`
- `vTaskDelay(pdMS_TO_TICKS(10))`

Nhờ vậy CPU vẫn nhường thời gian cho các task RTOS khác.

5.4 Đánh giá

Task 4 là bước chuyển quan trọng vì nó biến hệ thống nhúng thành một nút IoT có giao diện điều khiển thực tế. Từ đây, thiết bị không còn phụ thuộc vào serial monitor hoặc các nút vật lý trên board mà đã có thể tương tác qua trình duyệt. Đây cũng là nền tảng trực tiếp để về sau bổ sung dashboard giàu dữ liệu hơn ở Task 5 và Task 6.

6 Task 5: TinyML - huấn luyện, đánh giá và nhúng mô hình

6.1 Mục tiêu

Task 5 nhằm thay thế luật phân loại viết tay bằng một mô hình TinyML đủ nhỏ để chạy trực tiếp trên vi điều khiển. Mục tiêu cụ thể gồm:

- Tạo tập dữ liệu nhiệt độ, độ ẩm cho bài toán phân loại trạng thái môi trường.
- Huấn luyện một mạng nơ-ron gọn nhẹ.
- Xuất trọng số thành `tinymodel.h` để firmware có thể suy luận trực tiếp.
- Đo lại độ chính xác trên thiết bị thật thay vì chỉ đánh giá trên máy tính.

6.2 Mô tả dataset và mô hình

Theo tài liệu docs/task5_tinyml.md và docs/task5_metrics.json, bộ dữ liệu gồm:

- Tổng cộng 4800 mẫu
- 3840 mẫu train và 960 mẫu test
- Hai đặc trưng đầu vào: temperature_c và humidity_percent
- Ba nhãn đầu ra: normal, warning, critical

Dataset được tạo theo hai nguồn:

- **Uniform samples:** phủ miền giá trị hoạt động chung.
- **Boundary samples:** tăng mật độ mẫu quanh các ngưỡng quyết định để mô hình học tốt hơn ở vùng biên.

Kiến trúc mô hình được chọn là:

$$2 \rightarrow 10 \rightarrow 8 \rightarrow 3$$

với hàm kích hoạt ReLU và lớp ra softmax.

6.3 Kết quả huấn luyện

Bảng 4: Chỉ số đánh giá TinyML của Task 5

Chỉ số	Giá trị
Train accuracy	99.06%
Test accuracy	99.38%
Class accuracy – normal	97.56%
Class accuracy – warning	99.35%
Class accuracy – critical	99.65%
On-device replay accuracy	99.38%
Average inference latency	9.86 μ s
Số mẫu replay trên thiết bị	960

Ma trận nhầm lẫn trên tập test:

$$\begin{bmatrix} 80 & 2 & 0 \\ 0 & 306 & 2 \\ 0 & 2 & 568 \end{bmatrix}$$

Các kết quả trên cho thấy mô hình đủ tốt để nhúng lên thiết bị thật mà vẫn đảm bảo suy luận gần như tức thời.

6.4 Tích hợp vào firmware

Trong các nhánh hoàn thiện của Task 5, mô hình được nhúng dưới dạng file header `include/tinymml_model.h`. Sau khi cảm biến tạo ra một mẫu mới, dữ liệu sẽ đi qua pipeline:

- `sensorTask` đọc `temp/hum`
- `inferenceTask` chuẩn hóa dữ liệu, chạy `runTinyMLInference()`
- kết quả gồm `state` và `confidence`
- dữ liệu suy luận được dùng để cập nhật LCD, web dashboard và các vùng hiển thị khác

Một điểm mạnh của Task 5 là nhóm không dừng ở việc “train được model”, mà còn có hàm `evaluateTinyMLOnDevice()` để replay toàn bộ 960 mẫu kiểm thử ngay trên vi điều khiển. Điều này chứng minh rằng mô hình không chỉ đúng trên máy tính mà còn đúng sau khi đã chuyển đổi sang mã suy luận C/C++ trên board.

6.5 Nhận xét

Task 5 là bước nâng cấp mang tính học thuật lẫn kỹ thuật:

- Về học thuật, bài toán đã có quy trình đầy đủ: tạo dữ liệu, huấn luyện, đánh giá, nhúng, kiểm chứng lại.
- Về kỹ thuật, TinyML được đặt vào đúng vị trí trung tâm của hệ thống, thay vì chỉ là một minh họa rời rạc.

7 Task 6: Publish dữ liệu lên CoreIOT Cloud Server

7.1 Mục tiêu

Task 6 là phần mở rộng riêng của CO3037, đưa hệ thống từ mô hình dashboard cục bộ sang một thiết bị IoT có khả năng kết nối cloud:

- Kết nối WiFi ở chế độ AP + STA.
- Duy trì web dashboard cục bộ như ở Task 5.
- Kết nối MQTT đến `app.coreiot.io`.
- Publish telemetry định kỳ lên CoreIOT.

7.2 Kiến trúc phần mềm

Ở nhánh `bt1-iot-credit_6`, hệ thống được tổ chức thành nhiều task hoạt động song song:

- `sensorTask`: đọc DHT20 theo chu kỳ 2 giây.
- `inferenceTask`: chạy TinyML và cập nhật snapshot toàn hệ thống.
- `displayTask`: hiển thị trạng thái trên LCD I2C.
- `deviceTask`: xử lý lệnh bật/tắt thiết bị nhận từ web dashboard.
- `cloudTask`: lo toàn bộ phần Station WiFi, MQTT và publish telemetry.
- `webServerTask`: phục vụ dashboard HTTP cục bộ.

Phần `DashboardSnapshot` đóng vai trò như một vùng trạng thái tổng hợp, nơi web dashboard và cloud task có thể đọc cùng một ảnh chụp hệ thống mà không cần truy cập trực tiếp vào nhiều biến rời rạc.

7.3 Access Point, Station và dashboard

Khác với Task 4 chỉ có Access Point, Task 6 chạy ở chế độ `WIFI_MODE_APSTA`. Thiết bị:

- vẫn phát mạng cục bộ `C03037_IOT` để người dùng truy cập dashboard,
- đồng thời dùng thông tin WiFi STA trong `include/cloud_credentials.h` để đi Internet.

Dashboard web của Task 6 được thiết kế phong phú hơn nhiều so với Task 4:

- hiển thị nhiệt độ, độ ẩm, trạng thái TinyML và độ tin cậy mô hình,
- hiển thị tình trạng LCD, AP, STA và CoreIOT,
- có lịch sử nhiệt độ/độ ẩm dạng biểu đồ SVG,
- cho phép bật/tắt `Status LED` và `RGB Beacon` qua API `/api/device`.

7.4 Telemetry gửi lên CoreIOT

Theo tài liệu `docs/task6-coreiot.md`, mỗi chu kỳ publish gửi các trường sau:

Bảng 5: Telemetry của Task 6

Trường	Ý nghĩa
temperature	Nhiệt độ đo được từ DHT20
humidity	Độ ẩm đo được từ DHT20
tinymml_confidence	Độ tin cậy của nhãn TinyML
tinymml_state	Trạng thái suy luận: normal / warning / critical
tinymml_accuracy	Độ chính xác đo được khi replay trên thiết bị
rsssi	Cường độ tín hiệu WiFi hiện tại

Bên cạnh telemetry, firmware còn publish một số attribute mô tả thiết bị như địa chỉ MAC, địa chỉ IP, SSID mạng đang kết nối và tên mô hình TinyML. Trong bản báo cáo này, thông tin nhạy cảm như token truy cập và mật khẩu WiFi được cố ý lược bỏ để tránh lộ cấu hình thật.

7.5 Kết quả thực nghiệm

Từ log serial của nhánh Task 6 có thể rút ra các điểm sau:

- LCD được phát hiện thành công tại địa chỉ 0x21.
- TinyML đạt lại **99.38%** trên 960 mẫu kiểm thử với độ trễ trung bình **9.86 μ s**.
- Access Point khởi động thành công với IP 192.168.4.1.
- Thiết bị kết nối được WiFi Station và thiết lập thành công kết nối MQTT tới CoreIOT.
- Telemetry được publish định kỳ, ví dụ log `Published #1 T=26.3 C, H=48.8%`.

Điều này cho thấy Task 6 không chỉ là “thêm thư viện cloud”, mà là một bước tích hợp hoàn chỉnh giữa cảm biến, suy luận, hiển thị cục bộ, điều khiển cục bộ và đồng bộ dữ liệu lên nền tảng IoT.

7.6 Đánh giá

Task 6 là phần mở rộng quan trọng của toàn bộ hệ thống:

- Nếu Task 1–5 tập trung vào RTOS, hiển thị và suy luận nhúng, thì Task 6 bổ sung lớp kết nối hệ thống ra bên ngoài.
- Thiết kế `cloudTask` tách biệt giúp phần cloud không làm ảnh hưởng đến chu kỳ đọc cảm biến và suy luận TinyML.
- Dashboard nội bộ và cloud telemetry cùng dùng chung snapshot nên dữ liệu nhất quán giữa hai giao diện.

8 Kết luận

Qua 6 task đã thực hiện, hệ thống được phát triển theo một lộ trình rõ ràng:

- **Task 1:** xây được nền RTOS và cơ chế semaphore đúng bản chất.
- **Task 2:** thêm lớp hiển thị trực quan cho độ ẩm bằng NeoPixel.
- **Task 3:** hình thành cơ chế phân loại trạng thái môi trường và hiển thị có cấu trúc.
- **Task 4:** bổ sung giao tiếp mạng cục bộ và điều khiển qua web.
- **Task 5:** đưa TinyML vào vòng lặp vận hành thực tế của hệ thống.
- **Task 6:** hoàn thiện hệ thống theo mô hình IoT cloud-connected.

Toàn bộ quá trình thực hiện cho thấy ba giá trị kỹ thuật nổi bật của đề án:

- xây dựng ứng dụng nhúng theo RTOS,
- triển khai suy luận TinyML trên edge device,
- và kết nối dữ liệu lên nền tảng cloud để hình thành một pipeline IoT hoàn chỉnh.

Trong tương lai, hệ thống có thể được mở rộng thêm bằng các hướng như:

- thay dataset tổng hợp bằng dữ liệu môi trường thực tế dài ngày,
- thêm điều khiển hai chiều từ cloud xuống thiết bị,
- bổ sung lưu trữ lịch sử và cảnh báo thời gian thực trên dashboard.